

Ingeniería de Datos II

Clase #2

18 de Marzo de 2026

Parte a (Distribución de datos)

Distribución de datos

Introducción

- Las bases de datos NoSQL están diseñadas para operar de forma distribuida.
- Dos mecanismos clave para lograr escalabilidad, disponibilidad y rendimiento son:
 - Fragmentación (Sharding)
 - Replicación
- Ambos conceptos permiten manejar grandes volúmenes de datos en múltiples nodos.

Distribución de datos

¿ Qué es la fragmentación (sharding) ?

- Técnica que divide los datos de una base en partes más pequeñas (fragmentos).
- Cada fragmento se almacena en diferentes nodos.
- Cada nodo contiene solo un subconjunto de los datos.
- Objetivo principal: **escalar horizontalmente**.

Distribución de datos

Fragmentación en No SQL

- En NoSQL, la fragmentación suele ser:
 - Automática
 - Transparente para la aplicación
- Los datos se distribuyen según una clave de fragmentación (shard key).
- Ejemplo:
 - Usuarios con ID 1–10000 → Nodo A
 - Usuarios con ID 10001–20000 → Nodo B

Fragmentación de datos

Usos principales

- Manejo de grandes volúmenes de datos (Big Data).
- Aplicaciones con alta tasa de escritura/lectura.
- Escenarios donde un solo servidor no es suficiente.
- Plataformas:
 - Redes sociales
 - E-commerce
 - IoT
 - Analítica en tiempo real

Fragmentación de datos

Principales ventajas

- Escalabilidad horizontal casi ilimitada
- Mejor uso de recursos de hardware
- Mayor rendimiento por paralelismo
- Permite crecer agregando nodos económicos

Fragmentación de datos

Principales desventajas

- Mayor complejidad de diseño
- Consultas distribuidas más costosas
- Dependencia crítica de la shard key
- Posibles desbalances de carga (hotspots)

Fragmentación de datos

Algunas estrategias

- Por rango
 - Datos similares juntos
 - Riesgo de hotspots
- Por hash
 - Distribución uniforme
 - Consultas por rango más difíciles
- Por ubicación geográfica
 - Cercanía al usuario
 - Menor latencia

Fragmentación de datos

¿ Qué es la replicación de datos ?

- Técnica que **duplica los datos** en múltiples nodos.
- Cada réplica contiene una **copia total** o **parcial** de la base.
- Objetivo principal: **alta disponibilidad** y **tolerancia a fallos**.

Fragmentación de datos

Replicación en No SQL

- Frecuente uso de arquitecturas:
 - Primary–Secondary
 - Multi-master
- Un nodo puede caer sin afectar el servicio.
- Común en bases distribuidas globalmente.

Replicación de datos

Usos principales

- Alta disponibilidad (24/7).
- Lecturas escalables (read replicas).
- Tolerancia a fallos de hardware.
- Recuperación ante desastres (Disaster Recovery).

Replicación de datos

Principales ventajas

- Alta disponibilidad
- Tolerancia a fallos
- Mejor rendimiento en lecturas
- Soporte para sistemas distribuidos geográficamente

Replicación de datos

Principales desventajas

- Consumo extra de almacenamiento
- Mayor latencia en escrituras
- Problemas de consistencia
- Posibles conflictos en sistemas multi-master

Replicación de datos

Algunas estrategias

- **Síncrona**
 - Escritura confirmada en todas las réplicas
 - Alta consistencia, mayor latencia
- **Asíncrona**
 - Escritura primero en nodo primario
 - Mejor rendimiento, menor consistencia

Distribución de datos

Fragmentación vs replicación

Aspecto	Fragmentación	Replicación
Escalabilidad	👍 Si	👎 Limitada
Disponibilidad	👎 No directa	👍 Si
Duplicación de datos	👎 No	👍 Si
Complejidad	Alta	Media

Distribución de datos

Relación con el Teorema CAP

- Fragmentación
 - Prioriza Disponibilidad (A) y Tolerancia a particiones (P)
- Replicación
 - Mejora Disponibilidad (A)
 - Puede sacrificar Consistencia (C)
- En NoSQL es común elegir AP o CP.

Distribución de datos

Ejemplos en bases NoSQL reales

- **MongoDB**
 - Sharding + Replica Sets
- **Cassandra**
 - Distribución automática + replicación configurable
- **Redis Cluster**
 - Fragmentación por hash slots + replicación
- **DynamoDB**
 - Fragmentación y replicación totalmente administradas

Distribución de datos

Conclusión

- Fragmentación y replicación son **complementarias**, no excluyentes.
- Fragmentación → **escala**
- Replicación → **disponibilidad**
- NoSQL prioriza **distribución, elasticidad y resiliencia**.
- La elección depende del **caso de uso y requerimientos del sistema**.



Practiquemos un poco ...